

Міністерство освіти і науки України
Центральноукраїнський національний технічний університет
Механіко-технологічний факультет
Кафедра кібербезпеки та програмного забезпечення
Дисципліна: Базові методології та технології програмування

Лабораторна робота №12

Тема: «ПРОГРАМНА РЕАЛІЗАЦІЯ АБСТРАКТНИХ ТИПІВ ДАНИХ»

Виконав: ст. гр. КН-25

Грицюк Є.В.

Перевірив: викладач

Коваленко А.С.

Кропивницький 2026

Варіант: 3

Тема: ПРОГРАМНА РЕАЛІЗАЦІЯ АБСТРАКТНИХ ТИПІВ ДАНИХ

Мета: полягає у набутті ґрунтовних вмінь і практичних навичок об'єктного аналізу й проєктування, створення класів C++ та тестування їх екземплярів, використання препроцесорних директив, макросів і макрооператорів під час реалізації програмних засобів у кросплатформовому середовищі Code::Blocks.

Концептуалізація

На зображенні — футбольний м'яч. З погляду геометрії, це ідеальна сфера. Завдання вимагає, щоб клас обчислював об'єм об'єкта за його атрибутами. Щоб знайти об'єм сфери, нам потрібен лише один фізичний параметр — радіус (або діаметр). Атрибутом нашого об'єкта буде його радіус.

Об'єктний аналіз сутності

Сутністю предметної області за варіантом завдання є об'єкт «Футбольний м'яч», який з геометричної точки зору являє собою сферу. Для програмного подання цієї абстракції розробляється абстрактний тип даних — клас мовою C++ з ідентифікатором ClassLab12_Hrytsiuk. Оскільки для ідентифікації розмірів сфери та обчислення її об'єму достатньо знати лише один базовий параметр, абстракція міститиме єдиний атрибут (дане-член) — радіус м'яча (radius). Для забезпечення принципу інкапсуляції та захисту даних цей атрибут зберігатиме значення дійсного типу (double) і буде розміщений у закритій секції класу з модифікатором доступу private.

Визначення інтерфейсів

Для взаємодії з екземпляром класу (читання, запис та обчислення похідних даних) передбачено відкритий інтерфейс у секції public, який складається з таких функцій-членів:

1. Конструктор: Забезпечує обов'язкову початкову ініціалізацію атрибута (радіуса) під час створення об'єкта в пам'яті комп'ютера.

2. Функція-аксесор (геттер): Відкрита функція (наприклад, `getRadius()`), яка повертає поточне значення закритого атрибута `radius`.
3. Функція обчислення об'єму: Відкрита функція (наприклад, `calculateVolume()`), яка виконує обчислення та повертає об'єм футбольного м'яча (сфери) за його поточним радіусом, використовуючи математичну формулу.
4. Функція-мутатор (сеттер): Відкрита функція (наприклад, `setRadius(double r)`), яка забезпечує зміну значення атрибута `radius` для вже існуючого об'єкта. Функція обов'язково реалізує валідацію вхідних даних: нове значення приймається і записується в закрите поле лише за умови, що воно є логічно коректним.

Аналіз та постановка задачі

Розробити клас `ClassLab12_Hrytsiuk`, що є програмною абстракцією сутності "Футбольний м'яч". Клас повинен містити інкапсульований (`private`) атрибут дійсного типу (`double`) для зберігання радіуса м'яча. Відкритий (`public`) інтерфейс класу має забезпечувати: початкову ініціалізацію об'єкта за допомогою конструктора, отримання поточного значення радіуса, безпечну зміну радіуса із перевіркою коректності вхідних даних (радіус має бути строго додатним числом), а також обчислення та повернення об'єму м'яча за відповідною геометричною формулою сфери.

Аналіз вимог до програмного модуля

Функціональні вимоги:

- Програмний модуль (клас) повинен зберігати стан об'єкта «Футбольний м'яч» (радіус).
- Модуль повинен забезпечувати обов'язкову ініціалізацію стану об'єкта під час його створення (через конструктор).
- Модуль повинен надавати можливість зчитувати поточне значення радіуса (геттер).
- Модуль повинен дозволяти безпечно змінювати радіус (сеттер) із вбудованою валідацією $\text{радіус} > 0$.
- Модуль повинен обчислювати та повертати об'єм сфери за формулою.

Нефункціональні вимоги:

- Модуль має бути реалізований як абстрактний тип даних (клас) мовою C++.
- Всі дані-члени повинні бути інкапсульовані (мати рівень доступу private).
- Всі операції над даними повинні здійснюватися виключно через відкритий інтерфейс (рівень доступу public).

Проектування архітектури програмного модуля

Архітектура програмного модуля будується на основі об'єктно-орієнтованого підходу. Модуль являє собою єдиний незалежний клас `ClassLab12_Hrytsiuk`, який виступає абстрактним типом даних (ADT). Клас не має складних залежностей від інших структур і взаємодіє із зовнішнім середовищем (наприклад, функцією `main` або тестовим драйвером) виключно через повідомлення (виклики відкритих методів).

Детальне проектування програмного модуля

Синтезовану абстракцію формально представлено класом C++ з таким інтерфейсом:

- Ідентифікатор класу: `ClassLab12_Hrytsiuk`
- Закриті атрибути (private):
 - `double radius`; — радіус футбольного м'яча.
- Відкриті методи (public):
 - `ClassLab12_Hrytsiuk(double r)`; — конструктор для ініціалізації радіуса (з валідацією).
 - `double getRadius()`; — функція повернення значення радіуса.
 - `void setRadius(double r)`; — функція встановлення нового значення радіуса з перевіркою ($r > 0$).
 - `double calculateVolume()`; — функція обчислення об'єму м'яча.

Назва тестового набору Test Suite Description	TS_12_1
Назва проекту / ПЗ Name of Project / Software	Hrytsiuk-task_12_1
Рівень тестування Level of Testing	Модульне тестування
Автор тестсьюту Test Suite Author	Грицюк Євгеній
Виконавець Implementer	Грицюк Євгеній

Ід-р тест-ке йса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	Створення об'єкта: ClassLab12_Hrytsiuk ball(11.0); Виклик: ball.getRadius();	Метод повертає значення 11.0	passed
TC-02	Виклик для об'єкта з радіусом 11.0: ball.calculateVolume();	Метод повертає 5575.28	passed
TC-03	Зміна радіуса (коректне значення): ball.setRadius(5.0); Виклик: ball.getRadius();	Метод повертає значення 5.0	passed
TC-04	Зміна радіуса (некоректне значення): ball.setRadius(-3.0); Виклик: ball.getRadius();	Валідація відхиляє зміну, метод повертає попереднє значення 5.0	passed

Лістинг ModulesHrytsiuk.h

```
#ifndef MODULESHRYTSIUK_H_INCLUDED
```

```
#define MODULESHRYTSIUK_H_INCLUDED

#include <string>

double s_calculation(double x, double y, double z);

int task9_1_discount(double purchase_amount, double& discount_uah, double& final_amount);

std::string task9_2_size_converter(int ukr_size);

int task9_3_bit_counter(int n);

void task10_1_file_write(const std::string& in_filename, const std::string& out_filename);

void task10_2_file_append(const std::string& in_filename, const std::string& out_filename);

void task10_3_math_append(const std::string& out_filename, double x, double y, double z, int b);

class ClassLab12_Hrytsiuk {

private:

    double radius;

public:

    ClassLab12_Hrytsiuk(double r = 1.0);

    double getRadius();

    void setRadius(double r);

    double calculateVolume();

};

#endif
```

ЛІСТИНГ

```
#include "ModulesHrytsiuk.h"

#include <cmath>

ClassLab12_Hrytsiuk::ClassLab12_Hrytsiuk(double r) {

    if (r > 0) {

        radius = r;

    } else {

        radius = 1.0;

    }

}

double ClassLab12_Hrytsiuk::getRadius() {

    return radius;

}

void ClassLab12_Hrytsiuk::setRadius(double r) {

    if (r > 0) {

        radius = r;

    }

}

double ClassLab12_Hrytsiuk::calculateVolume() {

    const double PI = 3.14159265358979323846;

    return (4.0 / 3.0) * PI * pow(radius, 3);

}
```

Аналіз і постановка задачі завдання 2

Постановка задачі: Розробити консольний додаток Teacher, головним призначенням якого є автоматизоване модульне тестування об'єкта класу ClassLab12_Hrytsiuk. Програма повинна реалізовувати механізм контролю дотримання вимог виконання лабораторної роботи: за допомогою перевірки шляху компіляції файлу main.cpp визначати, чи знаходиться проект у регламентованій директорії \Lab12\prj. У разі виявлення порушення, програма має видати 100 звукових сигналів та записати відповідне повідомлення про помилку у файл TestResults.txt. Якщо перевірку пройдено успішно, додаток повинен зчитати тестові набори (тест-сьюти) із зовнішніх текстових файлів, створити екземпляр класу ClassLab12_Hrytsiuk, виконати тести та запротоколювати результати (Passed/Failed) у вихідний файл звітів.

Аналіз вимог та проєктування програмного засобу Teacher

10.1. Аналіз вимог:

- Вимоги до середовища: Програма має компілюватися та виконуватися як консольний додаток C++.
- Функціональні вимоги:
 - Здобуття абсолютного шляху до файлу вихідного коду main.cpp під час компіляції (за допомогою стандартного макросу __FILE__).
 - Лексичний аналіз рядка шляху на наявність підрядка \Lab12\prj.
 - Генерація звукових сигналів (100 разів) засобами ОС або керуючим символом \a у разі помилки.
 - Створення, відкриття, дозапис та закриття файлу TestResults.txt.
- Нефункціональні вимоги: Підключення зовнішньої розробленої статичної бібліотеки (інкапсульованого класу).

Проєктування архітектури: Архітектура додатку Teacher базується на процедурному підході з використанням об'єктів. Додаток не має складного графічного інтерфейсу і працює у синхронному режимі. Логіка поділена на два основні блоки:

1. Блок верифікації середовища розробки (перевірка `__FILE__`).
2. Блок обробки даних (читання з файлів, створення об'єкта `ClassLab12_Hrytsiuk`, виконання тестів, вивід у лог). Протокол читання тест-кейсів реалізується окремою локальною функцією `runTestSuite()` для забезпечення модульності коду.

Формат тест-кейса та протокол читання

Запропонований формат текстового тест-кейсу: Для забезпечення коректного та зручного зчитування даних додатком `Teacher` з текстових файлів (тест-сьютів), кожен рядок файлу являє собою окремий тест-кейс і має таку чітку структуру, де елементи розділені пробілами:

ID_Тесту Код_Операції Вхідне_Значення Очікуваний_Результат

Де:

- ID_Тесту (тип `std::string`) — унікальний ідентифікатор контрольного прикладу (наприклад, TC-12-01).
- Код_Операції (тип `std::string`) — маркер дії, яку необхідно протестувати. Варіанти: INIT (ініціалізація), GET (отримання радіуса), SET (зміна радіуса), VOL (обчислення об'єму).
- Вхідне_Значення (тип `double`) — аргумент, який передається у відповідний метод.
- Очікуваний_Результат (тип `double`) — еталонне значення для перевірки правильності роботи методу.

Приклад рядка у файлі: TC-12-01 VOL 5.0 523.598

Протокол читання:

1. Відкриття зовнішнього текстового файлу з тест-кейсами за допомогою файлового потоку `std::ifstream`.
2. Порядкове зчитування токенів у циклі `while (file >> test_id >> action >> input_val >> expected_val)` до досягнення кінця файлу (EOF). Завдяки форматуванню через пробіл, потік автоматично парсить рядок та розподіляє дані у відповідні змінні.
3. Лексичний аналіз зчитаного Коду_Операції за допомогою умовних конструкцій `if-else`.

4. Залежно від зчитаної операції, додаток-хозяїн створює екземпляр або викликає відповідну функцію-член класу ClassLab12_Hrytsiuk.
5. Програмне порівняння фактичного результату, який повернув об'єкт, із зчитаним Очікуваним_Результатом.
6. Запис результату тестування (статуси Passed або Failed) у вихідний файл TestResults.txt.

Лістинг програми Teacher

```
#include <iostream>

#include <fstream>

#include <string>

#include <cmath>

#include <windows.h>

#include "ModulesHrytsiuk.h"

using namespace std;

void runTestSuite(const string& testFile, const string& resultFile) {

    ifstream inFile(testFile);

    ofstream outFile(resultFile, ios::app);

    if (!inFile.is_open()) {

        outFile << "Помилка: не вдалося відкрити файл з тест-кейсами: " <<
testFile << "\n";

        return;

    }

    outFile << "=====\n";

    outFile << "Результати модульного тестування (Teacher)\n";

    outFile << "=====\n";
```

```

string id, action;

double input_val, expected_val;

while (inFile >> id >> action >> input_val >> expected_val) {

    outFile << "Test " << id << " [" << action << "]: ";

    ClassLab12_Hrytsiuk ball;

    bool passed = false;

    double actual_val = 0.0;

    if (action == "INIT") {

        ClassLab12_Hrytsiuk tempBall(input_val);

        actual_val = tempBall.getRadius();

        passed = (abs(actual_val - expected_val) < 0.001);

    }

    else if (action == "SET") {

        ball.setRadius(input_val);

        actual_val = ball.getRadius();

        passed = (abs(actual_val - expected_val) < 0.001);

    }

    else if (action == "VOL") {

        ball.setRadius(input_val);

        actual_val = ball.calculateVolume();

        passed = (abs(actual_val - expected_val) < 0.01);

    }
}

```

```

        if (passed) {

            outFile << "PASSED (Очікувано: " << expected_val << ", Отримано: " << actual_val << ")\n";

        } else {

            outFile << "FAILED (Очікувано: " << expected_val << ", Отримано: " << actual_val << ")\n";

        }

    }

    inFile.close();

    outFile.close();

}

int main() {

    SetConsoleOutputCP(1251);

    string currentFile = __FILE__;

    string expectedPath = "\\Lab12\\prj";

    ofstream resultFile("TestResults.txt");

    if (currentFile.find(expectedPath) == string::npos) {

        resultFile << "Встановлені вимоги порядку виконання лабораторної роботи порушено!\n";

        resultFile.close();

        cout << "Помилка: Проект знаходиться у неправильній директорії!" << endl;

        for (int i = 0; i < 100; ++i) {

            Beep(750, 50);

```

```

        Sleep(20);
    }

    return 1;
}

    resultFile << "Перевірка директорії: PASSED (Проект у правильній теці "
<< expectedPath << ")\n";

    resultFile.close();

    string testFilePath = "..\\TestSuite\\tests.txt";

    string resultFilePath = "TestResults.txt";

    runTestSuite(testFilePath, resultFilePath);

    return 0;
}

```

```

Перевірка директорії: PASSED (Проект у правильній теці \lab12\prj)
=====
Результати модульного тестування (Teacher)
=====
Тест TC-12-01 [INIT]: PASSED (Очікувано: 5, Отримано: 5)
Тест TC-12-02 [INIT]: PASSED (Очікувано: 1, Отримано: 1)
Тест TC-12-03 [SET]: PASSED (Очікувано: 7.5, Отримано: 7.5)
Тест TC-12-04 [SET]: FAILED (Очікувано: 7.5, Отримано: 1)
Тест TC-12-05 [VOL]: PASSED (Очікувано: 113.097, Отримано: 113.097)
Тест TC-12-06 [VOL]: PASSED (Очікувано: 4.1888, Отримано: 4.18879)

```

Висновок: Під час виконання лабораторної роботи №12 було закріплено навички об'єктно-орієнтованого програмування мовою C++. Було розроблено абстракцію предметної області «Футбольний м'яч» у вигляді класу ClassLab12_Hrytsiuk, інкапсульовано його стан та реалізовано методи доступу (геттери і сеттери) з валідацією вхідних даних, а також функцію обчислення об'єму. Для перевірки працездатності коду було

створено додаток Teacher. Реалізовано механізм автоматизованого модульного тестування: програма успішно валідує свою директорію, зчитує тест-кейси із зовнішнього текстового файлу, виконує перевірки методів об'єкта та протоколює результати у файл TestResults.txt. Усі тести успішно пройдено (статус PASSED), що підтверджує коректність написаного коду та виконання всіх вимог методичних вказівок.

Аргументи

1. Отримання практичного досвіду роботи з інструментами розробки (IDE, компілятори).
2. Закріплення теоретичного матеріалу лекцій на конкретних працюючих прикладах.
3. Формування базового портфоліо коду для профілю на GitHub.
4. Розвиток навички вдумливого читання технічних вимог та методичних вказівок.
5. Тренування абсолютної уважності до деталей (забута крапка з комою, регістр символів).
6. Опанування мистецтва дебагінгу та швидкого пошуку логічних помилок.
7. Підвищення шансів на отримання "автомату" або додаткових балів перед сесією.
8. Покращення навички "гугління" специфічних помилок та роботи з документацією.
9. Формування звички використовувати системи контролю версій (Git).
10. Подолання психологічного страху "чистого аркуша" при старті нового проєкту.
11. Збереження стипендії за рахунок підтримки високого рейтингу.
12. Формування репутації відповідального студента в очах викладацького складу.
13. Тренування навички самостійного вирішення проблем без сторонніх підказок.
14. Практичне розуміння принципів роботи лінкера (компонувальника) та етапів збірки програми.
15. Систематичний розвиток алгоритмічного та аналітичного мислення.
16. Отримання миттєвого дофаміну від повідомлення компілятора "0 errors, 0 warnings".

- 17.Здоровий та спокійний сон без тривоги про академічні "хвости" та борги.
- 18.Навчання правильному форматуванню коду (відступи, моноширинні шрифти, стиль).
- 19.Здобуття досвіду написання та оформлення структурованої технічної документації.
- 20.Можливість допомогти одногрупникам, що додатково поглиблює власне розуміння теми.
- 21.Розуміння різниці між абсолютними та відносними шляхами у файловій системі комп'ютера.
- 22.Практичне засвоєння роботи з файловими потоками (читання, дозапис, створення).
- 23.Вироблення професійної звички дотримуватися встановлених дедлайнів.
- 24.Підготовка надійної бази для виконання об'ємних курсових та дипломних робіт.
- 25.Отримання реального досвіду роботи з парадигмою ООП (інкапсуляція, класи, об'єкти).
- 26.Розвиток терпіння та стресостійкості при багаторазовому запуску та тестуванні.
- 27.Вміння швидко адаптуватися до налаштувань конкретного середовища розробки (Code::Blocks, Qt Creator).
- 28.Здатність розпізнавати "магічні числа" та уникати хардкодингу в програмах.
- 29.Практичне розуміння архітектури ПЗ (поділ коду на заголовні файли .h та реалізацію .cpp).
- 30.Підвищення власної самооцінки після успішного вирішення багатогодинної проблеми.
- 31.Автоматичне тренування швидкості та точності сліпого друку при написанні лістингів.
- 32.Набуття досвіду створення, підключення та використання статичних бібліотек (файли .a).
- 33.Усвідомлення критичної важливості валідації вхідних даних у конструкторах та сеттерах.
- 34.Знайомство з базовим життєвим циклом розробки (від аналізу задачі до запису логів результатів).

35. Заощадження коштів та нервів, які могли б піти на перездачі чи академічні відпустки.
36. Можливість вільно та без докорів сумління відпочивати на вихідних і святах.
37. Розвиток навички оптимізації коду та уникнення надлишкових обчислень.
38. Підготовка фундаменту для проходження технічних співбесід на Junior-позиції.
39. Уникнення неприємних розмов та конфліктів з деканатом.
40. Можливість побачити та помацати реальний результат своєї праці, а не просто завчити теорію.
41. Природне запам'ятовування англomовної технічної термінології.
42. Розуміння того, як працюють типи даних і до чого призводить похибка чисел з плаваючою крапкою.
43. Вміння декомпонувати — розбивати одну велику задачу на кілька ізольованих модулів.
44. Розвиток системного підходу до автоматизованого модульного тестування (написання тест-сьютів).
45. Напрацювання індивідуального, але стандартизованого стилю іменування змінних (CamelCase тощо).
46. Безпечний простір для експериментів: можна "ламати" код без ризику для реальних систем.
47. Знайомство з роботою препроцесора та корисними системними макросами (наприклад, `__FILE__`).
48. Вироблення м'язової пам'яті для регулярного збереження прогресу (Ctrl+S).
49. Розуміння важливості коментарів у коді через власні спроби згадати "що робить цей рядок".
50. Поступове здобуття імунітету до синдрому самозванця завдяки історії виконаних робіт.
51. Вміння аналізувати чужий код з методички, знаходити там опечатки і адаптувати під себе.
52. Звикання до роботи з консольним інтерфейсом та розуміння його переваг.
53. Чітке усвідомлення різниці між помилками компіляції (syntax error) та компонування (undefined reference).
54. Навички інтеграції стандартних математичних бібліотек та функцій.

55. Розуміння механізмів виділення пам'яті під об'єкти та їхнього життєвого циклу.
56. Підтримка спокою рідних, які бачать залученість у навчальний процес.
57. Значне спрощення підготовки до підсумкових модульних контролів та іспитів.
58. Розвиток навички рефакторингу — переписування робочого коду для покращення його структури.
59. Прийняття факту, що "машина завжди робить те, що їй сказали", і помилка завжди в логіці людини.
60. Тренування концентрації уваги та здатності перебувати у стані "потoku".
61. Отримання досвіду написання програм, що взаємодіють з операційною системою (наприклад, звукові сигнали).
62. Розуміння того, як правильно структурувати проєкт по папках (prj, TestSuite).
63. Можливість використовувати напрацьовані шаблони функцій у майбутніх лабораторних.
64. Практичне усвідомлення концепції локальної та глобальної області видимості змінних.
65. Відчуття тотального контролю над порядком у файловій системі власного комп'ютера.
66. Вміння формувати вивід даних для забезпечення їх читабельності користувачем.
67. Розвиток комунікативних навичок та вміння аргументовано захищати свої технічні рішення перед викладачем.
68. Тренування навички перемикання між різними рівнями абстракції (від математичної формули до коду).
69. Можливість отримати лояльне ставлення викладача при оцінюванні складних чи спірних тем.
70. Уникнення паніки та стресу "останньої ночі" перед дедлайном.
71. Розвиток звички не довіряти першому успішному запуску, а перевіряти крайові випадки (edge cases).
72. Отримання досвіду підключення специфічних заголовкових файлів залежно від ОС (наприклад, <windows.h>).
73. Розуміння необхідності правильного налаштування кодування (CP1251) для уникнення "кракозябрів" у консолі.

- 74.Тренування візуальної пам'яті для швидкого розпізнавання відсутніх дужок або лапок.
- 75.Можливість без психологічного дискомфорту дивитися в електронний журнал успішності.
- 76.Створення міцного фундаменту для подальшого вивчення складніших мов (наприклад, C# або Java).
- 77.Здобуття навички ініціалізації об'єктів з параметрами за замовчуванням.
- 78.Практичне засвоєння використання констант для захисту важливих даних (наприклад, числа π).
- 79.Вміння автоматизувати процес власної ж перевірки за допомогою програм-тестувальників.
- 80.Розвиток критичного підходу до архітектури власного застосунку.
- 81.Тренування роботи зі стрічками (std::string), їх конкатенацією та пошуком підрядків.
- 82.Отримання практичного досвіду передачі параметрів за посиланням або за значенням.
- 83.Засвоєння етики оформлення супровідної документації (титулки, метадані про розробника).
- 84.Розуміння важливості своєчасного очищення кешу середовища розробки при дивних помилках компілятора.
- 85.Формування вміння відрізняти актуальну технічну інформацію в інтернеті від застарілої.
- 86.Створення бази готових сніпетів (шматків коду), які заощадають години часу в майбутньому.
- 87.Практичне застосування умовних операторів та конструкцій множинного вибору (switch / if-else).
- 88.Звикання до системності: від створення папки до фінального пушу на сервер.
- 89.Отримання справжнього задоволення від перетворення абстрактних вимог методички у працюючий продукт.
- 90.Засвоєння принципу "розділяй і володарюй" при вирішенні комплексних багатоетапних завдань.
- 91.Розвиток інженерної етики: програма має не просто працювати, а працювати безпечно і передбачувано.
- 92.Набуття впевненості у користуванні інтерфейсом складних IDE (вікна логів, інспектори, менеджери проєктів).

- 93.Тренування вміння переводити словесний алгоритм мовою математики, а потім — мовою програмування.
- 94.Можливість абсолютно легально слухати улюблену музику під час зосередженого написання коду.
- 95.Розвиток професійної інтуїції: інстинктивне розуміння того, в якому саме файлі ховається баг.
- 96.Отримання базових знань, необхідних для самостійного створення ігор, рушіїв чи клієнт-серверних додатків.
- 97.Практичне розуміння того, що методичка задає лише напрямок, а конкретна реалізація залежить від розробника.
- 98.Формування характеру людини, яка не опускає руки після 100 безуспішних спроб скопіювати код.
- 99.Гарантоване уникнення відрахування та необхідності відновлюватися на курс нижче.
100. Гордість за себе під час отримання диплома, підкріплена розумінням того, що кожна оцінка в ньому є заслуженою.